student@pwr:~$ ./lab3.sh

Kontenery
Docker i containerd

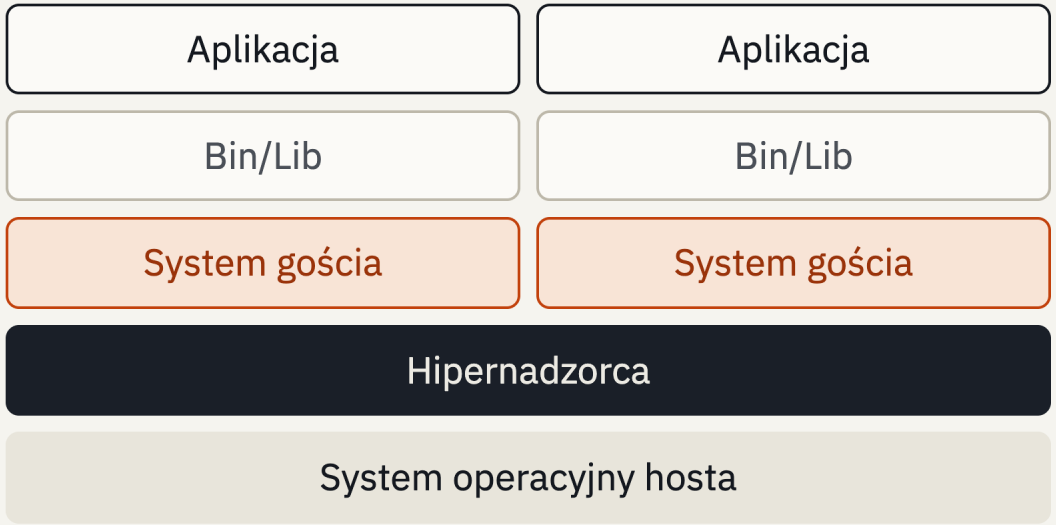
Czym kontener różni się od maszyny wirtualnej, co robi pod spodem jądro Linuksa i jak zbudować, opublikować i połączyć własne obrazy.

Przetwarzanie danych masowych · Laboratorium 3 · Politechnika Wrocławska

01 · TEORIA

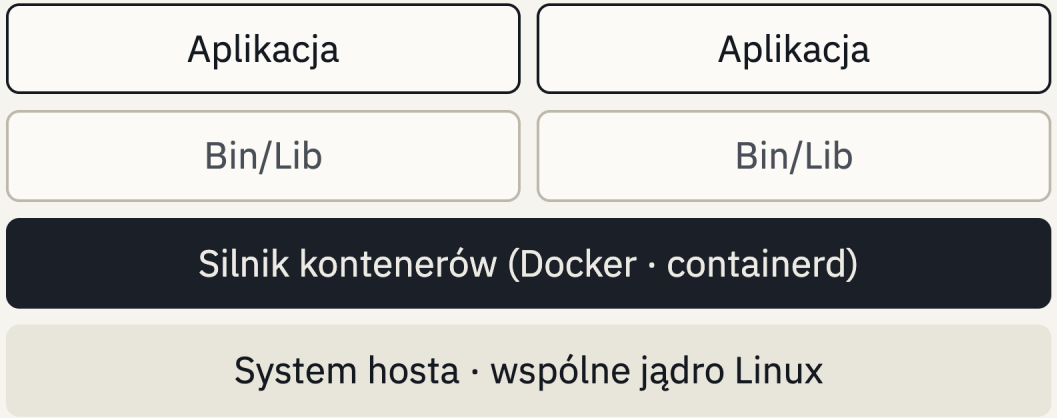
Maszyna wirtualna a kontener

Maszyna wirtualna



Każda VM uruchamia **pełny system z własnym jądrem**. Mocna izolacja, ale duży narzut i wolny start.

Kontener



Kontenery **współdzielą jądro hosta** i niosą tylko aplikację z bibliotekami. Lekkie, startują jak zwykły proces.

02 · TEORIA

Kontener to zwykły proces z ograniczeniami

namespaces · co proces widzi

Własne drzewo procesów, sieć, system plików i nazwa hosta.

`pid` `net` `mnt` `uts` `ipc` `user`

cgroups · ile proces może zużyć

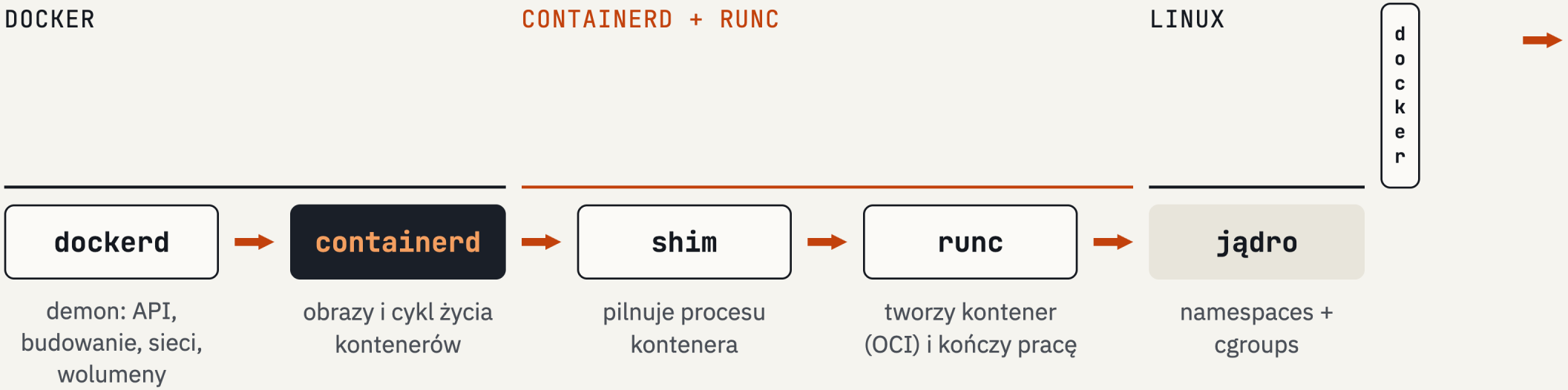
Limity CPU, pamięci, operacji dyskowych i liczby procesów.

`cpu` `memory` `io` `pids`

```
# cgroups: limit 256 MB RAM i pół rdzenia CPU
$ docker run -d --name web --memory 256m --cpus 0.5 nginx:alpine
# namespace pid: w kontenerze nginx ma PID 1
$ docker exec web ps
```

macOS i Windows: Docker Desktop uruchamia kontenery w lekkiej VM z Linuksem, bo potrzebują jądra Linux.

Docker, containerd, runc — kto co robi



Kubernetes i K3S

Rozmawiają z containerd bezpośrednio przez CRI, bez dockerd. Wrócimy do tego przy K3S.

Standard OCI

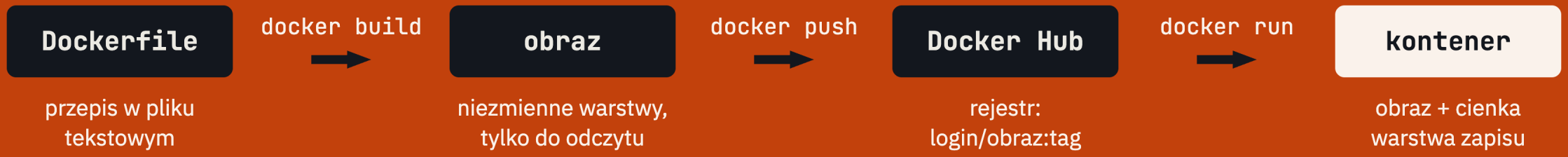
Wspólny format obrazów. Obraz z docker build uruchomisz w containerd, Podmanie i Kubernetesie.

CLI dla containerd

ctr do diagnostyki, nerdctl ze składnią jak w Dockerze:

```
nerdctl run -p 8080:80 nginx
```

Obraz to przepis zapisany w warstwach. Kontener to jego uruchomiona instancja.



Jeden obraz → wiele kontenerów. To, co kontener zapisze, znika po docker rm — trwałe dane trzymaj w wolumenie (-v).

05 · DOCKERFILE

Anatomia Dockerfile

```
ARG PY_VERSION=3.12                # zmienna tylko na czas budowania
FROM python:${PY_VERSION}-slim     # obraz bazowy
ENV GREETING="Cześć"               # zmienna widoczna w kontenerze
WORKDIR /app                       # katalog roboczy
RUN pip install requests           # wykonane przy budowaniu → warstwa
COPY app.py .                      # plik z kontekstu budowania
ENTRYPOINT ["python", "app.py"]    # co startuje z kontenerem…
CMD ["student"]                    # …a to? różnica → Zadanie 2
```

Warstwy i cache

Każde RUN/COPY/ADD to warstwa. Stabilne kroki daj wyżej.

.dockerignore

Wyklucz .git, venv i dane z kontekstu budowania.

Zadanie 2

ENTRYPOINT czy CMD? ADD czy COPY? Ile FROM? Sprawdź sami.

06 · CACHE

Kolejność kroków a cache

Najpierw kod, potem zależności

```
FROM python:3.12-slim      CACHED
WORKDIR /app               CACHED
COPY app.py .              ZMIANA
RUN pip install requests   OD NOWA
```

Zmiana app.py → pip install od nowa: **5,0 s**

Najpierw zależności, potem kod

```
FROM python:3.12-slim      CACHED
WORKDIR /app               CACHED
RUN pip install requests   CACHED
COPY app.py .              ZMIANA
```

Zmiana app.py → pip install z cache: **0,4 s**

Zasada

Pierwszy zmieniony krok unieważnia cache wszystkich kolejnych. Stabilne kroki — wyżej.

W praktyce

Najpierw COPY requirements.txt i pip install -r, dopiero potem COPY . . z kodem.

Klucz cache RUN

RUN porównuje tylko tekst polecenia. Nowe wersje z PyPI: docker build --no-cache.

Warstwy i rozmiar obrazu

Dwa polecenia RUN

```
RUN pip install requests
RUN rm -rf /root/.cache/pip
# znika… tylko z widoku
```

`RUN rm -rf …`	znacznik .wh.pip	**12 kB**
`RUN pip install`	requests + cache pip	**15 MB**
`python:3.12-slim`	obraz bazowy	

Obraz cięższy o **15 MB** (z pandas: 187 MB)

Jedno polecenie RUN

```
RUN pip install requests \
    && rm -rf /root/.cache/pip
# albo: pip install --no-cache-dir
```

`RUN pip … && rm`	tylko requests	**13,4 MB**
`python:3.12-slim`	obraz bazowy	

Obraz cięższy o **13,4 MB** (z pandas: 158 MB)

> **Warstwy są tylko do odczytu.** rm w kolejnej warstwie zapisuje jedynie znacznik .wh.pip, który ukrywa katalog. Bajty zostają niżej i jadą z każdym docker pull.

08 · CLI

Docker na co dzień

POLECENIE	DO CZEGO	PRZYKŁAD
build	zbuduj obraz z Dockerfile	`docker build -t hello:1.0 .`
run	uruchom kontener z obrazu	`docker run --rm -it hello:1.0`
ps	lista kontenerów (-a: wszystkie)	`docker ps -a`
logs	wyjście kontenera	`docker logs -f web`
exec	wejdź do działającego kontenera	`docker exec -it web sh`
images · rmi	lista · usuwanie obrazów	`docker rmi hello:1.0`
stop · rm	zatrzymaj · usuń kontener	`docker rm -f web`
tag · push	publikacja (najpierw docker login)	`docker push login/hello:1.0`

Pomoc: `docker run --help` Porządki: `docker system prune`

09 · PRZYKŁAD

Od Dockerfile do działającego kontenera

```
$ docker build -t hello:1.0 .
$ docker run --rm hello:1.0
Cześć, student!
$ docker run --rm -e GREETING=Hej hello:1.0
Hej, student!
$ docker run -d --name web -p 8080:80 nginx:alpine
$ curl -s localhost:8080 | grep title
<title>Welcome to nginx!</title>
$ docker rm -f web
```

`--rm`
usuń kontener po zakończeniu

`-e KLUCZ=wartość`
nadpisz zmienną z ENV

`-d · --name web`
w tle, pod czytelną nazwą

`-p 8080:80`
port komputera : port kontenera

Wiele kontenerów: docker compose

```yaml
# compose.yaml
services:
  api:
    build: .
    ports: ["8000:8000"]
    environment:
      REDIS_HOST: cache
    depends_on: [cache]
    restart: unless-stopped
  cache:
    image: redis:7-alpine
    restart: unless-stopped
```

```
$ docker compose up -d        # start w tle
$ docker compose ps           # stan usług
$ docker compose logs -f api  # logi usługi
$ docker compose down         # stop + sprzątanie
```

Sieć Compose

Usługi z jednego pliku dzielą sieć i widzą się po nazwie: api łączy się z cache:6379. links jest zbędne.

Aktualna składnia

Plik docker-compose.yml lub compose.yaml, bez przestarzałego pola version:. Polecenie: docker compose.

11 · ZADANIA

Do zrobienia w domu

ZADANIE 1 · 4 PKT

Własny Dockerfile

Kontener wypisuje powitanie z banerem ASCII. Użyj: FROM, RUN, ADD, ENV, ARG, ENTRYPOINT, CMD.

`Dockerfile`

ZADANIE 2 · 1 PKT

Cztery pytania

Różnice ENTRYPOINT/CMD i ADD/COPY, nadpisywanie przy starcie, kilka FROM w jednym pliku.

`zadanie2-odpowiedzi.md`

ZADANIE 3 · 1 PKT

Skrypt publish.sh

Buduje obraz z Dockerfile podanego jako argument i publikuje go na Docker Hub.

`publish.sh`

ZADANIE 4 · 4 PKT

Stos w Compose

content (np. NGINX ze stroną), load-balancer i checker (curl co 5 s). Polityka restart, sieć Compose, bez links.

`docker-compose.yml`

Razem 10 pkt. Pliki startowe są w katalogu lab3/. Przed Zadaniem 3 załóż konto na Docker Hub i zrób docker login.